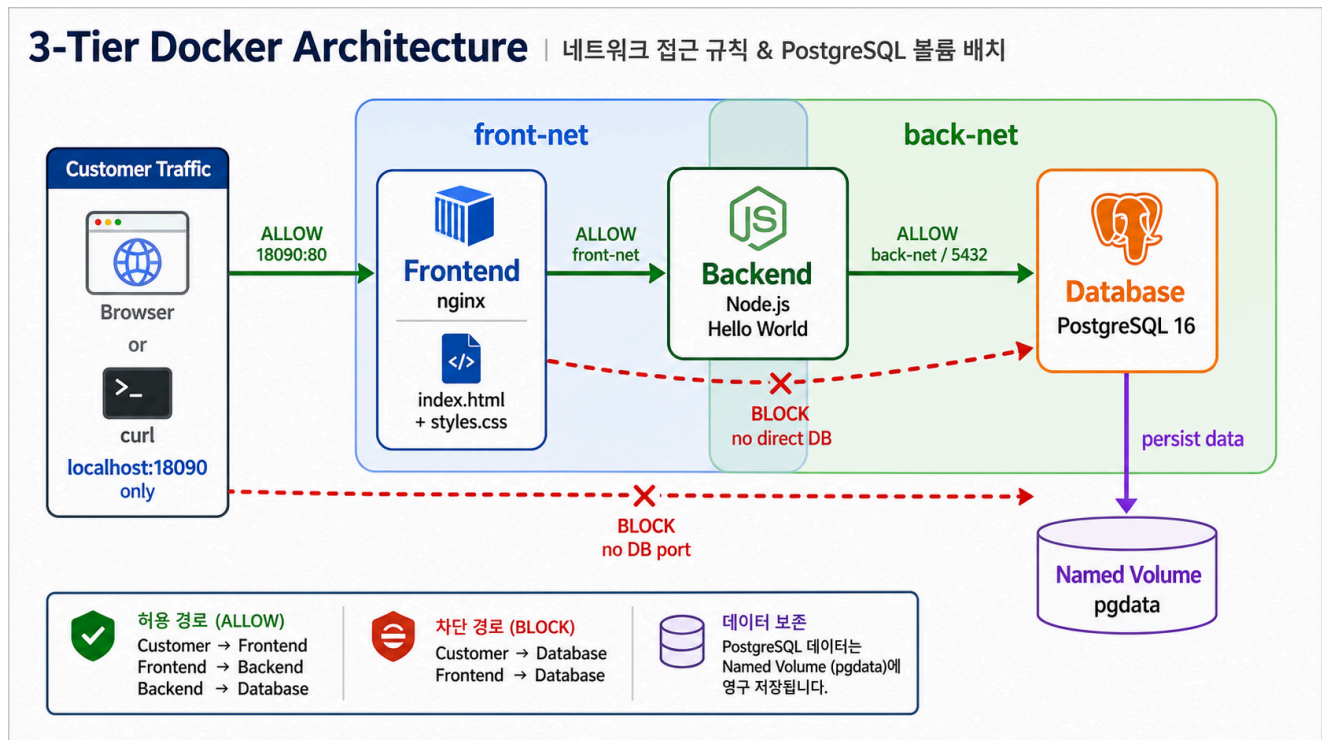


Session 09 Challenge: 3-Tier Image Delivery and Runtime Evidence

이 주말 챌린지는 Day 3의 image/build/tag 기준을 3-tier 구조로 확장한다. 목표는 멋진 기능을 많이 만드는 것이 아니라, frontend, backend, database를 분리해 실행하고 **네트워크, 볼륨, 로그, 트러블슈팅, 이미지 최적화 수치**를 증거로 남기는 것이다.

성공 기준은 의도적으로 단순하다. frontend는 index.html과 styles.css를 nginx로 제공한다. backend는 Node.js hello world API를 띄운다. database는 postgres:16 container와 named volume이 정상 생성되고 유지되는 것만 확인해도 성공이다.

Architecture



Tier	Container	Image	Network	역할
Frontend	paperclip-weekend-front	직접 build	paperclip-weekend-front-net	index.html, styles.css 제공, backend로 proxy
Backend	paperclip-weekend-back	직접 build	paperclip-weekend-front-net, paperclip-weekend-back-net	Node.js hello world API, DB network 확인
Database	paperclip-weekend-db	postgres:16	paperclip-weekend-back-net	PostgreSQL 16, named volume 사용

그림 해석 기준:

- Host는 frontend의 18090:80 publish port로만 들어간다.
- Frontend는 paperclip-weekend-front-net에만 붙고 backend만 호출한다.
- Backend는 front-net, back-net 양쪽에 붙는 유일한 application tier다.
- Database는 paperclip-weekend-back-net에만 붙고 host port를 publish하지 않는다.
- PostgreSQL data는 container filesystem이 아니라 paperclip-weekend-pgdata named volume에 남는다.

Network rule:

```
host -> frontend
frontend -> backend
backend -> database
frontend -X-> database
host -X-> database port
```

frontend는 backend와 같은 network에만 붙인다. database는 backend와 같은 network에만 붙인다. 따라서 frontend에서 database 이름을 바로 찾을 수 없어야 한다. ping만 성공해도 연결 증거로 인정하지만, 가능하면 HTTP 또는 port check도 함께 남긴다.

File Map

Path	역할
frontend/Dockerfile	frontend nginx image
assets/session-09-3tier-architecture.png	network, connection rule, volume 구조도
frontend/Dockerfile.size-compare	frontend base image size 비교
frontend/.dockerignore	frontend build context에서 .env, node_modules, output 제외
frontend/nginx.conf	/api/ 요청을 backend로 proxy
frontend/html/index.html	browser 확인용 정적 HTML
frontend/html/styles.css	browser 확인용 정적 CSS
backend/Dockerfile	backend Node.js hello world image
backend/Dockerfile.size-compare	backend base image size 비교
backend/.dockerignore	backend build context에서 node_modules, cache, secret 제외
backend/server.js	/health, /api/info 응답
scripts/measure-build.sh	최초 build 시간과 image size 측정
SUBMISSION.md	제출 보고서 템플릿

Lab root:

```
cd /mnt/d/paperclip/week2/day3/labs/weekend-3tier-challenge
```

1. Build

```
cd /mnt/d/paperclip/week2/day3/labs/weekend-3tier-challenge
docker build -t paperclip-weekend-frontend:optimized ./frontend
docker build -t paperclip-weekend-backend:optimized ./backend
docker images 'paperclip-weekend-*
```

Expected:

```
paperclip-weekend-frontend    optimized
paperclip-weekend-backend    optimized
```

2. Create network and volume

```
docker network create paperclip-weekend-front-net || true
docker network create paperclip-weekend-back-net || true
docker volume create paperclip-weekend-pgdata
docker network ls | grep paperclip-weekend
docker volume ls | grep paperclip-weekend
```

Expected:

```
paperclip-weekend-front-net
paperclip-weekend-back-net
paperclip-weekend-pgdata
```

3. Run DB

```
docker rm -f paperclip-weekend-db || true
docker run -d \
  --name paperclip-weekend-db \
  --network paperclip-weekend-back-net \
  -e POSTGRES_PASSWORD=weekend-only \
  -e POSTGRES_DB=weekend \
  -v paperclip-weekend-pgdata:/var/lib/postgresql/data \
```

```
postgres:16
```

```
docker logs paperclip-weekend-db --tail 40
```

Expected:

```
database system is ready to accept connections
```

4. Run backend

```
docker rm -f paperclip-weekend-back || true
```

```
docker run -d \
```

```
--name paperclip-weekend-back \
```

```
--network paperclip-weekend-back-net \
```

```
-e APP_ENV=weekend \
```

```
-e DB_HOST=paperclip-weekend-db \
```

```
-e DB_PORT=5432 \
```

```
paperclip-weekend-backend:optimized
```

```
docker network connect paperclip-weekend-front-net paperclip-weekend-back
```

```
docker logs paperclip-weekend-back --tail 40
```

Expected:

```
node backend listening on 8080
```

5. Run frontend

```
docker rm -f paperclip-weekend-front || true
```

```
docker run -d \
```

```
--name paperclip-weekend-front \
```

```
--network paperclip-weekend-front-net \
```

```
-p 18090:80 \
```

```
paperclip-weekend-frontend:optimized
```

```
curl -I http://localhost:18090
```

```
curl -s http://localhost:18090/api/info
```

Expected:

```
HTTP/1.1 200 OK
"service": "node-backend"
```

6. Network evidence

Frontend to backend must work. ping 성공만으로도 network 연결 증거로 인정하고, HTTP는 application path 확인으로 추가한다.

```
docker exec paperclip-weekend-front ping -c 2 paperclip-weekend-back
docker exec paperclip-weekend-front wget -q -O- http://paperclip-weekend-back:8080/health
```

Expected:

```
2 packets transmitted
ok
```

Backend to DB must work. ping만 성공해도 인정한다.

```
docker exec paperclip-weekend-back ping -c 2 paperclip-weekend-db
docker exec paperclip-weekend-back sh -c 'nc -z paperclip-weekend-db 5432 && echo db-port-open'
```

Expected:

```
2 packets transmitted
db-port-open
```

Frontend to DB must fail:

```
docker exec paperclip-weekend-front ping -c 2 paperclip-weekend-db || true
docker exec paperclip-weekend-front wget -q -O- http://paperclip-weekend-db:5432 || true
```

Expected failure:

```
bad address 'paperclip-weekend-db'
```

Interpretation:

frontend는 front-net에만 있다.
db는 back-net에만 있다.
backend만 두 network에 모두 붙어 있으므로 frontend -> backend, backend -> db 흐름만 허용된다.

7. Volume evidence

```
docker volume inspect paperclip-weekend-pgdata
docker logs paperclip-weekend-db --tail 20
```

Expected:

```
"Name": "paperclip-weekend-pgdata"
database system is ready to accept connections
```

추가 확인:

```
docker rm -f paperclip-weekend-db
docker run -d \
  --name paperclip-weekend-db \
  --network paperclip-weekend-back-net \
  -e POSTGRES_PASSWORD=weekend-only \
  -e POSTGRES_DB=weekend_changed \
  -v paperclip-weekend-pgdata:/var/lib/postgresql/data \
  postgres:16
docker logs paperclip-weekend-db --tail 40
```

Expected signal:

```
Database directory appears to contain a database; Skipping initialization
```

Interpretation:

named volume에 기존 DB data가 남아 있으므로 POSTGRES_DB를 바꿔도 최초 초기화가 다시 실행되지 않는다.

8. Logs and troubleshooting evidence

```
docker logs paperclip-weekend-front --tail 40
docker logs paperclip-weekend-back --tail 40
```

```
docker logs paperclip-weekend-db --tail 40
docker ps --filter name=paperclip-weekend
docker inspect paperclip-weekend-back --format '{{json
.NetworkSettings.Networks}}'
```

RCA mini drill:

Symptom	First evidence command	Likely cause	Fix
frontend /api/info returns 502	docker logs paperclip-weekend-front	backend not on front network	docker network connect paperclip-weekend-front-net paperclip-weekend-back
backend cannot reach DB	docker exec paperclip-weekend-back ping ...	backend not on back network or DB name wrong	connect backend to back network, check container name
DB data does not reset	docker logs paperclip-weekend-db	stale named volume	decide whether to remove volume
build is slow or image is large	scripts/measure-build.sh	base image or context size	compare alpine/slim/default, check .dockerignore

9. Build speed and image size measurement

최초 build 속도와 image size를 반드시 숫자로 남긴다. cache가 섞이면 수치가 흐려지므로 --no-cache로 측정한다. 처음 실행하는 machine에서는 base image pull 시간이 포함될 수 있다. 그래서 제출할 때는 처음 pull 포함, 이미 pull된 상태 중 어느 조건인지 함께 적는다.

Build context hygiene check:

```
sed -n '1,120p' frontend/.dockerignore
sed -n '1,120p' backend/.dockerignore
du -sh frontend backend
```

Expected:

.env, .env.*, node_modules, dist, build, __pycache__, .venv 같은 파일/디렉터리가 제외 규칙에 있다.

```
chmod +x scripts/measure-build.sh
./scripts/measure-build.sh
```

Expected output:

```
target,base,seconds,size_bytes,size_human
frontend,nginx:stable,...
frontend,nginx:stable-alpine,...
frontend,nginx:stable-trixie,...
backend,node:22,...
backend,node:22-slim,...
backend,node:22-alpine,...
```

제출 표에는 다음을 기록한다.

Target	Base image	First build seconds	Size	선택 여부	이유
frontend	nginx:stable				
frontend	nginx:stable-alpine				
frontend	nginx:stable-trixie				
backend	node:22				
backend	node:22-slim				
backend	node:22-alpine				

Image size가 커지면 push/pull 시간, registry storage, runner/node pull time, 취약점 scan 시간이 늘어날 수 있다. 가장 작은 image가 항상 정답은 아니지만, 선택 근거는 숫자로 설명해야 한다.

10. Upload or submission

Docker Hub push는 선택이다. push한다면 public/private 범위와 secret 포함 여부를 먼저 확인한다.

```
docker tag paperclip-weekend-frontend:optimized <dockerhub-id>/paperclip-weekend-frontend:week2-day3
docker tag paperclip-weekend-backend:optimized <dockerhub-id>/paperclip-weekend-backend:week2-day3
# docker push <dockerhub-id>/paperclip-weekend-frontend:week2-day3
# docker push <dockerhub-id>/paperclip-weekend-backend:week2-day3
```

Push하지 않는 경우에도 SUBMISSION.md를 작성해 제출한다.

Required evidence:

1. architecture table
2. network evidence: frontend -> backend success, backend -> db success, frontend -> db fail
3. volume evidence: pgdata inspect and stale volume interpretation

4. logs: frontend/backend/db key lines
5. troubleshooting RCA 1개 이상
6. build speed and image size table
7. image tag or optional registry URL

Cleanup

```
docker rm -f paperclip-weekend-front paperclip-weekend-back paperclip-weekend-  
db || true  
docker network rm paperclip-weekend-front-net paperclip-weekend-back-net ||  
true  
  
# data reset이 필요할 때만 실행한다.  
# docker volume rm paperclip-weekend-pgdata  
  
# image까지 지울 때만 실행한다.  
# docker image rm paperclip-weekend-frontend:optimized paperclip-weekend-  
backend:optimized
```